MODEXP Circuit

개요

MODEXP Circuit은 EVM의 모듈러 지수 연산(Modular Exponentiation) 프리컴파일 호출을 검증하는 회로입니다. 이 회로는 `misc-precompiled-circuit` 크레이트의 `ModExpChip` 을 기반으로 하며, 큰 정수(최대 256비트)에 대한 모듈러 지수 연산 `base^exp mod modulus` 를 검증합니다.

주요 특징:

- 최대 256비트(32바이트) 정수 지원
- 108비트 limb 분해를 통한 효율적인 큰 수 처리
- Range check를 통한 limb 유효성 검증
- ModExp 테이블을 통한 EVM Circuit과의 통합
- 각 연산당 약 39,962행 사용

파일 위치:

- `zkevm-circuits/src/modexp_circuit.rs` - 메인 회로 구조
- `zkevm-circuits/src/table.rs` - ModExpTable 정의
- `zkevm-circuits/src/evm_circuit/execution/precompiles/modexp.rs` - EVM 프리컴파일 통합

모듈러 지수 연산 개요

모듈러 지수 연산은 다음 수식을 계산합니다:

```
result = base^exponent mod modulus
```

이더리움 프리컴파일

이더리움의 MODEXP 프리컴파일(주소 0x05)은:

- 임의 크기의 큰 정수 지원
- 가변 길이 입력 (base, exponent, modulus)
- RSA 서명 검증 등에 사용

Scroll의 구현 제약

현재 구현은 다음 제약을 가집니다:

- 최대 256비트(32바이트) 정수 지원
- 고정된 회로 크기 (연산당 약 39,962행)
- U256 타입으로 제한

회로 구조

ModExpCircuitConfig

`ModExpCircuitConfig` 는 MODEXP Circuit의 설정을 정의합니다:

```
pub struct ModExpCircuitConfig {
modexp_config: ModExpConfig, // ModExpChip 설정
rangecheck_config: RangeCheckConfig, // Range check 설정
modexp_table: ModExpTable, // ModExp 테이블
}
```

컴포넌트:

1. **modexp_config**: `misc-precompiled-circuit` 의 `ModExpChip` 설정

- 모듈러 지수 연산의 핵심 로직
- 큰 수 산술 연산 처리

2. **rangecheck_config**: Range check 칩 설정

- Limb 값이 유효 범위 내에 있는지 검증
- 108비트 limb의 오버플로우 방지

3. **modexp_table**: ModExp 테이블

- EVM Circuit과의 데이터 공유
- 입력(base, exp, modulus)과 출력(result) 저장

ModExpTable 구조

ModExpTable은 모듈러 지수 연산의 입출력을 저장합니다:

```
pub struct ModExpTable {
q_head: Column<Fixed>, // Limb 그룹 시작 표시
base: Column<Advice>, // Base 값 (limbs)
exp: Column<Advice>, // Exponent 값 (limbs)
modulus: Column<Advice>, // Modulus 값 (limbs)
result: Column<Advice>, // Result 값 (limbs)
}
```

컬럼 역할:

1. **q_head**: Limb 그룹의 시작을 표시하는 Fixed 컬럼

- 각 연산마다 4행 사용 (3개 limb + 1개 native)

- 첫 번째 행에서만 1, 나머지는 0

2. **base, exp, modulus, result**: 각각 Advice 컬럼

- 각 값은 4행에 걸쳐 저장
- 처음 3행: 108비트 limb 표현
- 마지막 행: Native field 표현

Number 표현

큰 정수는 `Number` 구조체로 표현됩니다:

```
pub struct Number<F> {
limbs: [Limb<F>; 4], // 3개 limb + 1개 native
}

pub struct Limb<F> {
cell: Option<AssignedCell<F, F>>, // 할당된 셀
value: F, // 값
}
```

Limb 분해 방식:

256비트 정수를 다음과 같이 분해:

- limb[0]: 하위 108비트
- limb[1]: 중간 108비트
- limb[2]: 상위 40비트
- limb[3]: Native field 표현 (modulo field order)

예시 (U256 값 분해):

```
value = 0x123456789ABCDEF0123456789ABCDEF0123456789ABCDEF0123456789ABCDEF0
```

```
limb[0] = value & ((1 << 108) - 1)

limb[1] = (value >> 108) & ((1 << 108) - 1)

limb[2] = (value >> 216) & ((1 << 108) - 1)

limb[3] = value mod field_order
```

데이터 흐름

1. 입력 데이터 수집

Bus-Mapping에서 `BigModExp` 이벤트 생성:

```
pub  struct BigModExp {

base: Word, // Base 값

exponent: Word, // Exponent 값

modulus: Word, // Modulus 값

result: Word, // 계산 결과

}
```

2. Witness 생성

`ModExpCircuit::new_from_block()` 에서:

- 블록에서 `BigModExp` 이벤트 수집
- 이벤트 수를 회로 용량에 맞게 조정
- 빈 이벤트로 패딩 (필요시)

```
let  event_limit = block.circuits_params.max_keccak_rows / MODEXPCONFIG_EACH_CHI
let  mut  exp_events = block.get_big_modexp();
if  event_limit != 0 {
assert!(exp_events.len() <= event_limit);
exp_events.resize(event_limit, Default::default());
}
```

3. 회로 할당

`assign_group()` 에서 각 연산 처리:

```
1. Base 값 할당 (4행)
↓
2. Exponent 값 할당 (4행)
↓
3. Modulus 값 할당 (4행)
↓
4. ModExpChip으로 연산 수행
↓
5. Result 값 할당 (4행)
```

```
↓

6. q_head 설정 (첫 행만 1)
```

4. 테이블 룩업

EVM Circuit에서:

- MODEXP 프리컴파일 호출 시
- ModExp 테이블에서 입력 값 룩업
- 결과 값 검증

제약 조건

1. Limb 분해 제약

```
// U256 값이 올바르게 limb로 분해되었는지 검증

value = limb[0] + limb[1] * 2^108 + limb[2] * 2^216
```

2. Range Check 제약

```
// 각 limb가 유효 범위 내에 있는지 검증

0 ≤ limb[0] < 2^108

0 ≤ limb[1] < 2^108

0 ≤ limb[2] < 2^108
```

3. Native Field 제약

```
// Native 표현이 올바른지 검증
limb[3] = value  mod field_order
```

4. 모듈러 지수 연산 제약

```
// ModExpChip에서 검증
result = base^exponent  mod modulus

// 제약 조건:
// 1. result < modulus
// 2. ∃ quotient: base^exponent = result + quotient * modulus
```

5. 테이블 일관성 제약

```
// q_head 제약
q_head[0] = 1
q_head[1] = 0
q_head[2] = 0
q_head[3] = 0

// Result limb가 ModExp 테이블에 복사됨
for  i  in  0..4:
```

```
table.result[offset + i] = result.limbs[i]
```

ModExpChip 알고리즘

`misc-precompiled-circuit` 의 `ModExpChip` 은 다음 알고리즘을 사용합니다:

1. Square-and-Multiply 알고리즘

모듈러 지수 연산의 기본 알고리즘:

```
function modexp(base, exp, mod):
result = 1
base = base mod mod
while exp > 0:
if exp is odd:
result = (result * base) mod mod
exp = exp >> 1
base = (base * base) mod mod
return result
```

2. 큰 수 산술

108비트 limb를 사용한 큰 수 연산:

덧셈:

```
// Limb별 덧셈 + carry 처리
```

```
(c0, s0) = limb_a[0] + limb_b[0]

(c1, s1) = limb_a[1] + limb_b[1] + c0

(c2, s2) = limb_a[2] + limb_b[2] + c1
```

곱셈:

```
// Limb별 곱셈 + carry 전파

// Karatsuba 알고리즘 사용 가능
```

모듈러 감소:

```
// Barrett reduction 또는 Montgomery reduction

// Quotient 계산 및 검증
```

3. Range Check 통합

모든 중간 결과에 대해 range check 수행:

- Limb 값이 108비트 범위 내
- Carry 값이 유효 범위 내
- 최종 결과가 modulus보다 작음

성능 특성

행 사용량

```
const MODEXPCONFIG_EACH_CHIP_ROWS: usize = 39962;
```

- 각 모듈러 지수 연산: 39,962행
- 테이블 행: 연산당 4행
- 총 행: 39,962 + 4 = 39,966행/연산

회로 용량

```
let event_limit = max_keccak_rows / MODEXPCONFIG_EACH_CHIP_ROWS;
```

- k=20 (1,048,576행): 약 26개 연산
- k=21 (2,097,152행): 약 52개 연산
- k=22 (4,194,304행): 약 104개 연산

증명 시간

- 단일 연산: 약 5-10초 (k=20 기준)
- 병렬 처리 가능 (여러 연산 동시 증명)

구현 세부사항

assign_group 메서드

```
pub(crate) fn assign_group<F: Field>(
&self,
region: &mut Region<F>,
table_offset: usize, // 테이블 행 오프셋
mut calc_offset: usize, // 계산 행 오프셋
```

```
event: &BigModExp, // 입력 이벤트
modexp_chip: &ModExpChip<F>,
range_check_chip: &mut RangeCheckChip<F>,
) -> Result<usize, Error>
```

처리 단계:

1. **Base 할당**

```
let base = self.assign_value(
region, table_offset,
self.modexp_table.base,
&event.base
)?;
```

2. **Exponent 할당**

```
let exp = self.assign_value(
region, table_offset,
self.modexp_table.exp,
&event.exponent
)?;
```

3. **Modulus 할당**

```
let modulus = self.assign_value(
```

```
region, table_offset,
self.modexp_table.modulus,
&event.modulus
)?;
```

4. **모듈러 지수 연산 수행**

```
let ret = modexp_chip.mod_exp(
region,
range_check_chip,
&mut calc_offset,
&base,
&exp,
&modulus,
)?;
```

5. **Result 복사 및 q_head 설정**

```
for i in 0..4 {
// q_head 설정
region.assign_fixed(
self.modexp_table.q_head,
table_offset + i,
if i == 0 { F::one() } else { F::zero() },
)?;
```

```
// Result limb 복사
ret.limbs[i].cell.copy_advice(
region,
self.modexp_table.result,
table_offset + i,
)?;
}
```

assign_value 메서드

U256 값을 limb로 분해하여 할당:

```
fn assign_value<F: Field>(
&self,
region: &mut Region<F>,
offset: usize,
col: Column<Advice>,
value: &Word,
) -> Result<Number<F>, Error>
```

처리 과정:

1. **Limb 분해**

```
let limbs_v = ModExpTable::split_u256_108bit_limbs(value);
```

```
// [limb0, limb1, limb2]
```

2. **Native 계산**

```
let native_v = ModExpTable::native_u256(value);
// value mod field_order
```

3. **Limb 할당**

```
for (i, limb) in limbs_v.into_iter().enumerate() {
let fv = F::from_u128(limb);
let c = region.assign_advice(col, offset + i, fv)?;
limbs.push(Limb::new(Some(c), fv));
}
```

4. **Native 할당**

```
let c = region.assign_advice(col, offset + 3, native_v)?;
limbs.push(Limb::new(Some(c), native_v));
```

synthesize_sub 메서드

전체 회로 합성:

```
fn synthesize_sub(
&self,
config: &Self::Config,
```

```
_challenges: &Challenges<Value<F>>,
layouter: &mut impl Layouter<F>,
) -> Result<(), Error>
```

처리 흐름:

1. **칩 초기화**

```
let modexp_chip = ModExpChip::new(config.modexp_config.clone());
let mut range_chip = RangeCheckChip::new(config.rangecheck_config.clone());
```

2. **리전 할당**

```
layouter.assign_region(|| "modexp circuit", |mut region| {
// Range chip 초기화
range_chip.initialize(&mut region)?;
// 각 이벤트 처리
let mut calc_offset = 0;
for (n, event) in self.0.iter().enumerate() {
calc_offset = config.assign_group(
&mut region,
n * 4, // 테이블 오프셋
calc_offset, // 계산 오프셋
event,
&modexp_chip,
```

```
&mut  range_chip,
)?;
}
Ok(())
})?;
```

3. **빈 행 채우기**

```
config.modexp_table.fill_blank(layouter)
```

EVM Circuit 통합

프리컴파일 호출

EVM Circuit의 MODEXP 프리컴파일 처리:

```
// 1. CallData에서 입력 파싱
let  base_len = read_u256(calldata, 0);
let  exp_len = read_u256(calldata, 32);
let  mod_len = read_u256(calldata, 64);

let  base = read_bytes(calldata, 96, base_len);
let  exp = read_bytes(calldata, 96 + base_len, exp_len);
let  modulus = read_bytes(calldata, 96 + base_len + exp_len, mod_len);
```

```
// 2. ModExp 테이블 룩업
lookup(modexp_table, base, exp, modulus) -> result


// 3. 결과를 ReturnData에 저장
write_bytes(return_data, 0, result);
```

가스 비용 계산

MODEXP의 가스 비용은 입력 크기에 따라 동적:

```
// EIP-2565 가스 계산
let  mult_complexity = max(base_len, mod_len)^2;
let  iteration_count = max(exp_len * 8, exp_bitlen);
let  gas_cost = mult_complexity * iteration_count / 3;
```

최적화 기법

1. Limb 크기 선택

108비트 limb 사용:

- 3개 limb로 256비트 표현 (108 + 108 + 40)
- Field 크기(254비트)보다 작아 오버플로우 방지
- 곱셈 시 carry 처리 간소화

2. Native Field 표현

각 값의 native field 표현 저장:

- Field 산술 연산 최적화
- Equality 제약 간소화
- RLC 계산 효율화

3. Range Check 재사용

RangeCheckChip을 여러 연산에서 공유:

- 룩업 테이블 재사용
- 회로 크기 감소
- 증명 시간 단축

4. 고정 회로 크기

각 연산에 고정된 행 수 할당:

- 회로 크기 예측 가능
- 증명 생성 안정성
- 병렬 처리 용이

제한사항 및 향후 개선

현재 제한사항

1. **크기 제한**: 최대 256비트 정수만 지원
- 이더리움 프리컴파일은 임의 크기 지원
- 큰 RSA 키 등 처리 불가
2. **고정 비용**: 모든 연산에 동일한 행 수 사용

- 작은 지수에도 많은 행 소비
- 회로 효율성 저하

3. **순차 처리**: 연산을 순차적으로 처리

- 병렬 처리 제한적
- 증명 시간 증가

향후 개선 방향

1. **가변 크기 지원**

- 512비트, 1024비트 등 확장
- 동적 limb 수 할당

2. **적응형 회로**

- 지수 크기에 따른 행 수 조정
- 작은 연산의 효율성 개선

3. **배치 처리**

- 여러 작은 연산을 하나의 리전에서 처리
- 회로 활용도 향상

4. **알고리즘 최적화**

- Montgomery ladder 사용
- Sliding window 기법 적용
- 사전 계산 테이블 활용

구현 예시

단일 연산 검증

```rust
// BigModExp 이벤트 생성
let event = BigModExp {
base: U256::from(3),
exponent: U256::from(5),
modulus: U256::from(13),
result: U256::from(9), // 3^5 mod 13 = 243 mod 13 = 9
};

// 회로 생성
let circuit = ModExpCircuit(vec![event], PhantomData);

// 증명 생성
let prover = MockProver::run(k, &circuit, vec![])?;
prover.assert_satisfied();
```

여러 연산 검증

```rust
let events = vec![
BigModExp {
base: U256::from(2),
exponent: U256::from(10),
modulus: U256::from(1000),
result: U256::from(24), // 2^10 mod 1000 = 1024 mod 1000 = 24
```

```
},
BigModExp {
base: U256::from(7),
exponent: U256::from(3),
modulus: U256::from(100),
result: U256::from(43), // 7^3 mod 100 = 343 mod 100 = 43
},
];

let circuit = ModExpCircuit(events, PhantomData);
```

큰 수 연산

```
let base = U256::from_dec_str(
"1234567890123456789012345678901234567890123456789 0"
).unwrap();
let exp = U256::from(65537); // RSA 공개 지수
let modulus = U256::from_dec_str(
"9876543210987654321098765432109876543210987654321 0"
).unwrap();

// 실제 계산 (외부 라이브러리 사용)
let result = base.pow_mod(exp, modulus);
```

```
let  event = BigModExp {
base,
exponent: exp,
modulus,
result,
};
```

관련 문서

- SHA256 Circuit - 다른 프리컴파일 회로
- ECC Circuit - 타원곡선 프리컴파일
- EVM Circuit - 프리컴파일 호출
- 테이블 시스템 - 테이블 개념

참고 자료

- EIP-198 - MODEXP 프리컴파일 명세
- EIP-2565 - MODEXP 가스 비용 개선
- misc-precompiled-circuit - ModExpChip 구현
- Montgomery Modular Multiplication - 최적화 알고리즘